



IAD-014 Redes Neurais

aula 1

Introdução às Redes Neurais Artificiais

Claudio Pinhanez

(a partir de materiais criados por Marlon Sproesser Mathias)



Claudio Pinhanez

40 anos como cientista de IA
professor, inovador, consultor

CIAAM – Centro de IA e Aprendizado de Máquina da USP
Professor Visitante (a partir de 1º de junho, 2026)

2009-2025 IBM Research Brasil
Principal Research Scientist

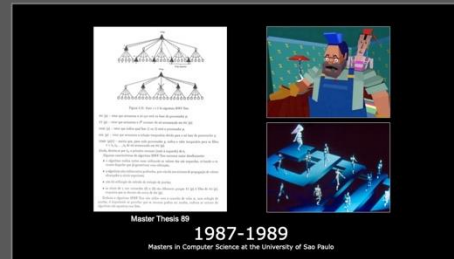
2020-2025 C4AI – USP-IBM-FAPESP
Vice-Diretor

1999-2009 IBM Research T. J. Watson
Research Scientist

1987-1993 University of São Paulo
Professor Assistente

1993-1999 PhD MIT Media Laboratory
1985-1987 Mestrado em Computação USP
1982-1985 Bacharel em Matemática USP





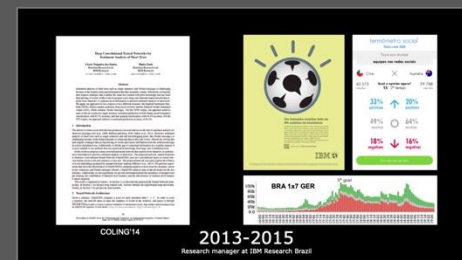
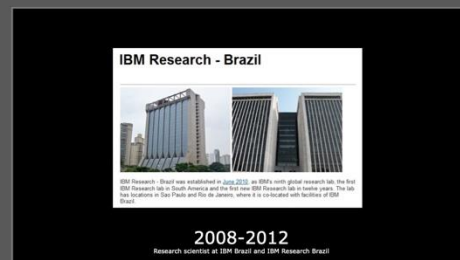
Bacharelado e Mestrado, Universidade de São Paulo

Universidade de Osaka

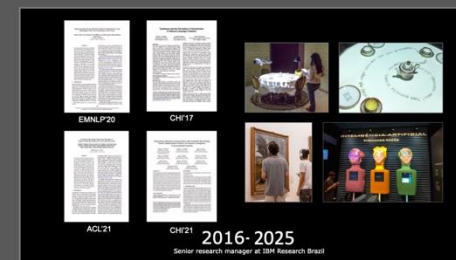


PhD, MIT Media Laboratory

IBM Research Watson



IBM Research Brasil



Universidade de São Paulo

Ementa

- Conceitos básicos e introdução às redes neurais artificiais.
- Treinamento supervisionado e não supervisionado.
- Principais arquiteturas de redes.
- Redes “alimentadas-adiante”.
- Perceptrons, unidades lineares e não lineares.
- Redes rasas e redes profundas.
- O algoritmo de retro-propagação.
- Ajuste de funções e classificação de padrões usando redes neurais artificiais.
- Introdução à plataforma TensorFlow/Keras.
- Criação, treinamento e avaliação de redes neurais sequenciais usando o Keras.
- Configuração de conjunto de dados
- Inicialização dos parâmetros da rede.
- Métodos de otimização.

Calendário e Trabalhos

Julho: 30

Agosto: 06, 13, 20, 27

Setembro: 03, 10, 17, 24

Outubro: 01

Horário: 19:30 – 22:30 (15min break)

	enunciado	entrega
Trabalho 1:	06 de agosto	20 de agosto
Trabalho 2:	27 de agosto	10 de setembro
Trabalho 3:	17 de setembro	01 de outubro

Média final: $(2 \cdot \text{participação} + 2 \cdot T1 + 3 \cdot T2 + 3 \cdot T3) / 10$

Uso de IA nos trabalhos:

O uso extensivo de IA nos trabalhos não é permitido nem aconselhável.

Erros estúpidos e básicos, típicos de IA, serão altamente penalizados.

O aluno/a deverá ser capaz de responder integralmente pelo que foi submetido sob pena de anulação.

Bibliografia

- Goodfellow, I., Bengio, Y., Courville, A. Deep Learning.
<https://www.deeplearningbook.org/>
- Chollet, F. Deep Learning with Python, Manning
Shelter Island Editor, 2018.
- Jurafsky, D. & Martin, J., Speech and Language
Processing, 3rd. Ed. Draft, chapter 7,
<https://web.stanford.edu/~jurafsky/slp3/7.pdf> , 2020.

Conhecendo vocês

- Nome
- Formação
- Atuação profissional atual
- Conhecimentos/uso de IA

- Por que está fazendo o MBA?

Aula 1 – Introdução às Redes Neurais Artificiais

Histórico
Inteligência
Artificial

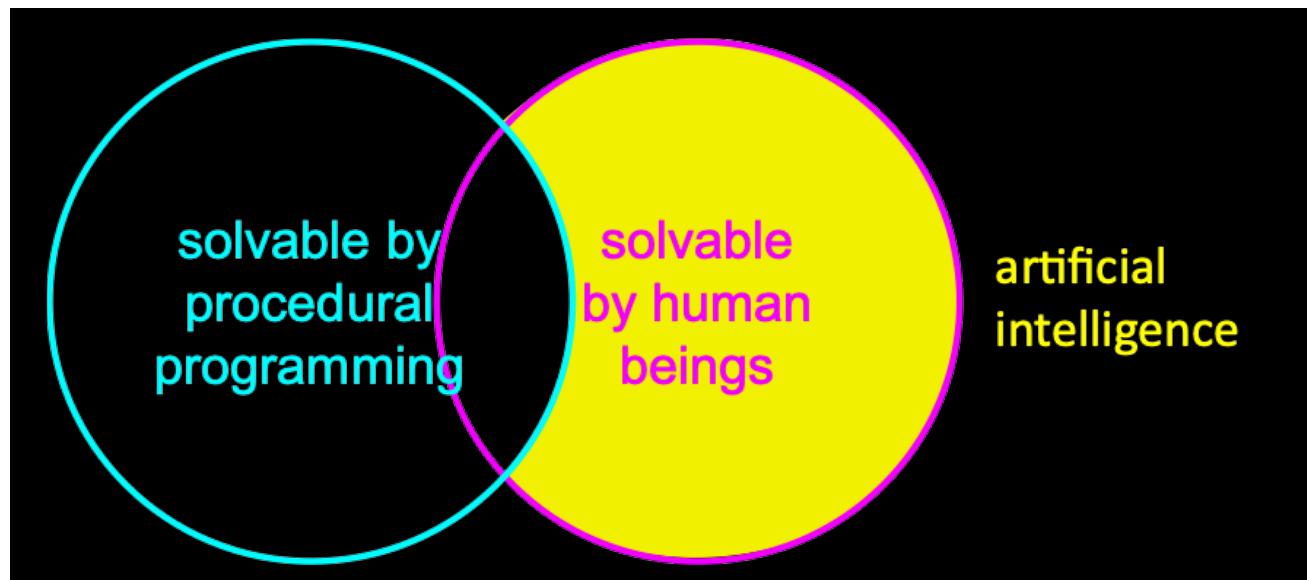
Redes Neurais
Artificiais

Estrutura de
uma RNA

Treinamento
supervisionado
e não-
supervisionado

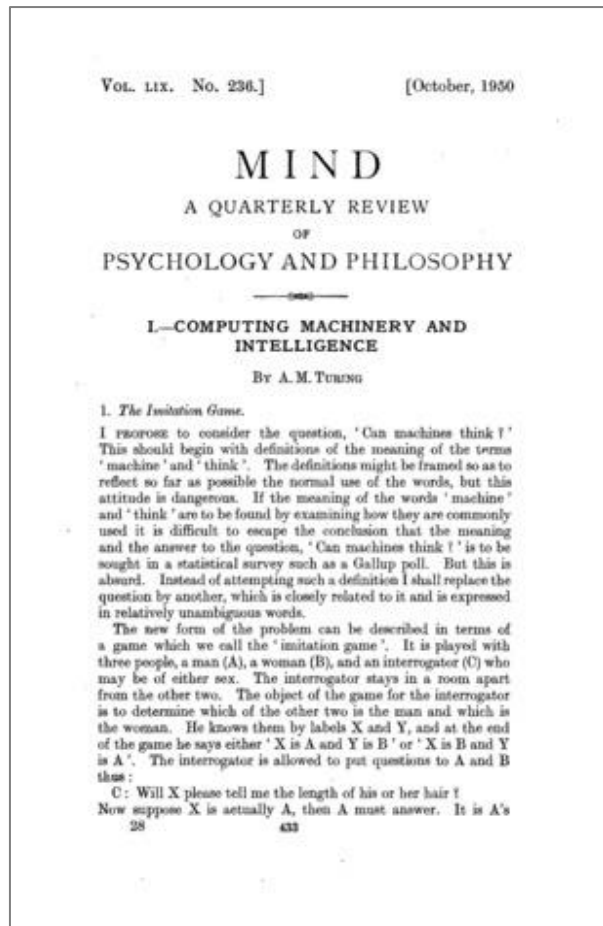
Histórico da Inteligência Artificial

A minha definição do que é Inteligência Artificial (IA)

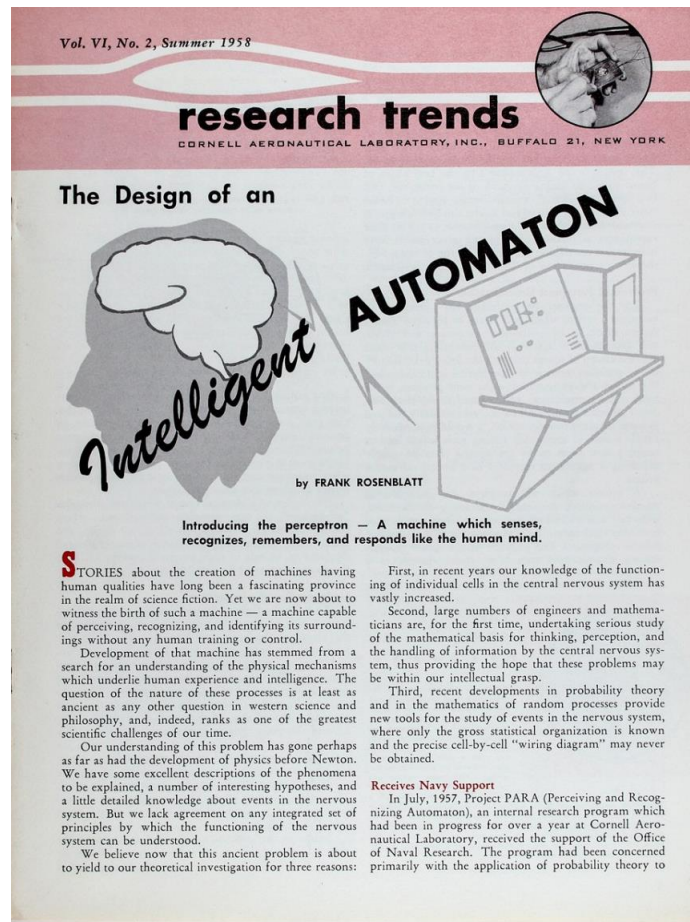


IA é a classe de problemas solucionáveis por humanos que não são bem resolvidos por programação procedural

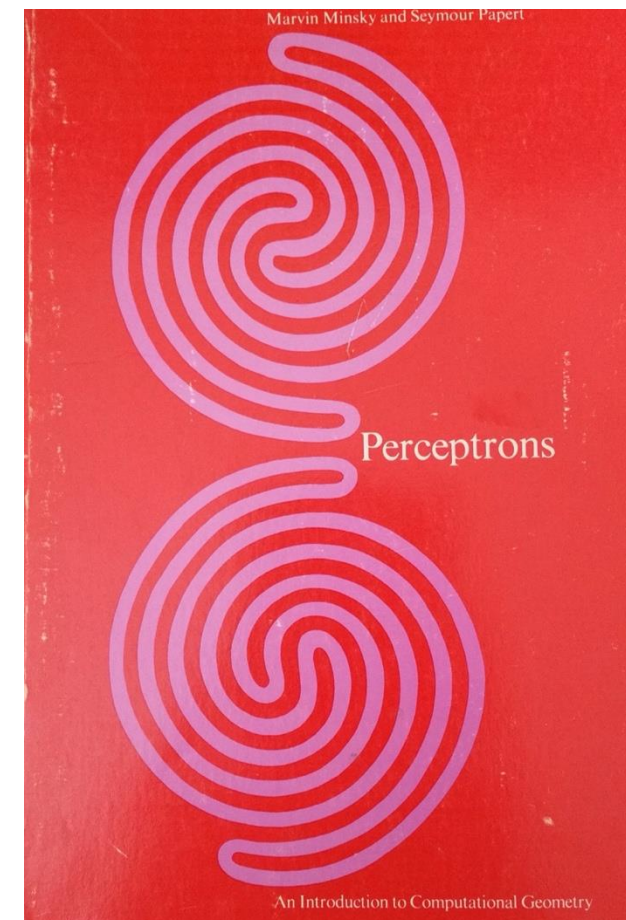
IA até a década de 1980



Turing's Imitation Game

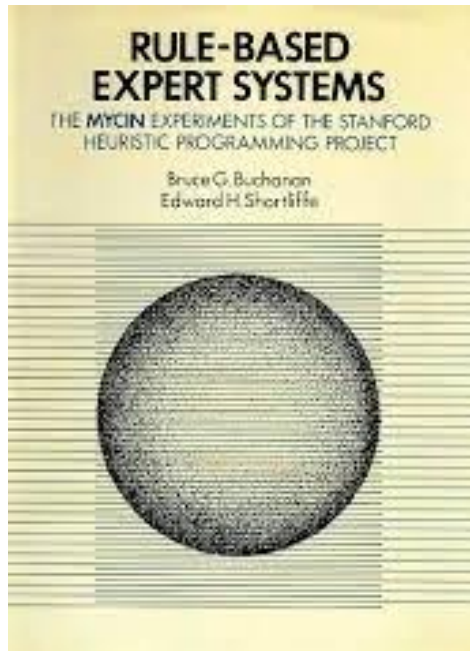


Rosemblatt's perceptrons

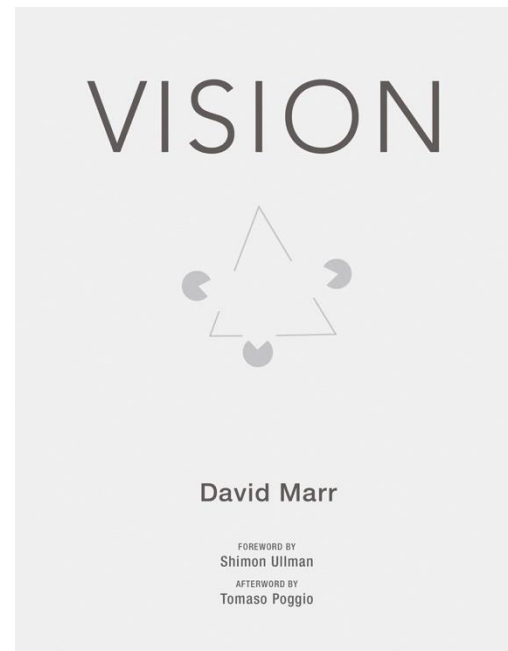


Minsky/Papert's proof

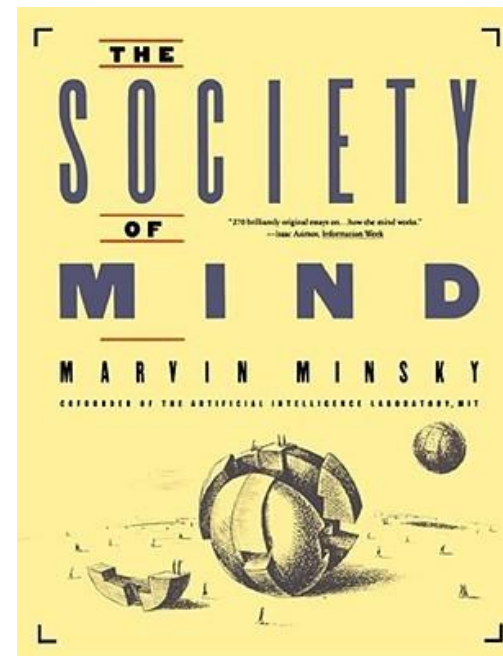
IA no início dos anos 1980



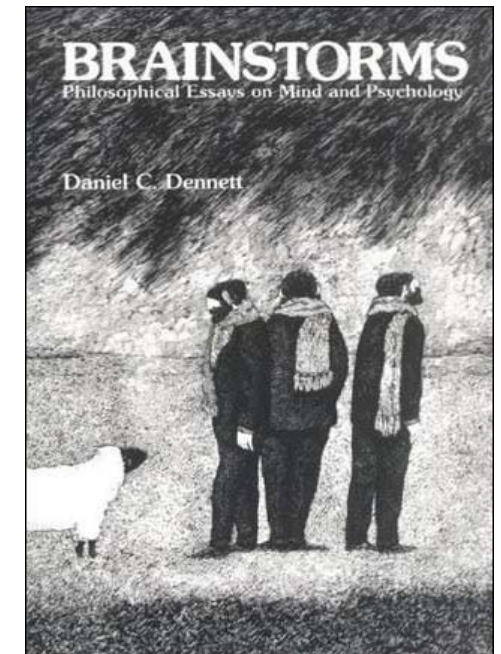
MYCIN's rules



Marr's 2D to 3D vision

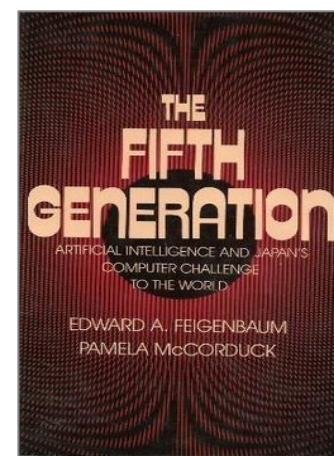
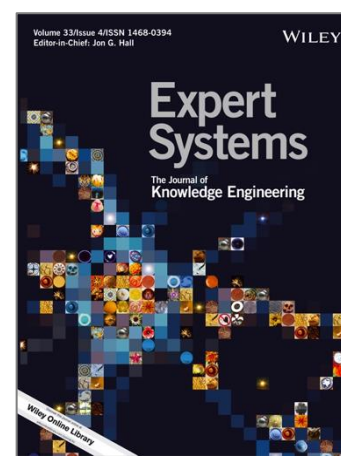
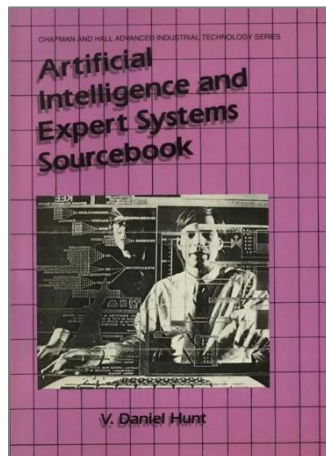
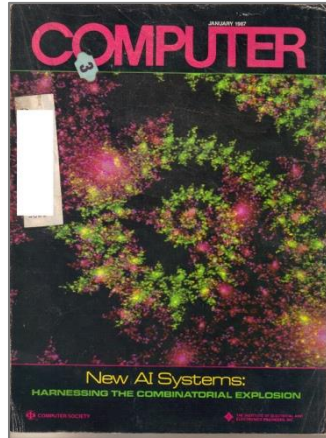


Minsky's Society of Mind

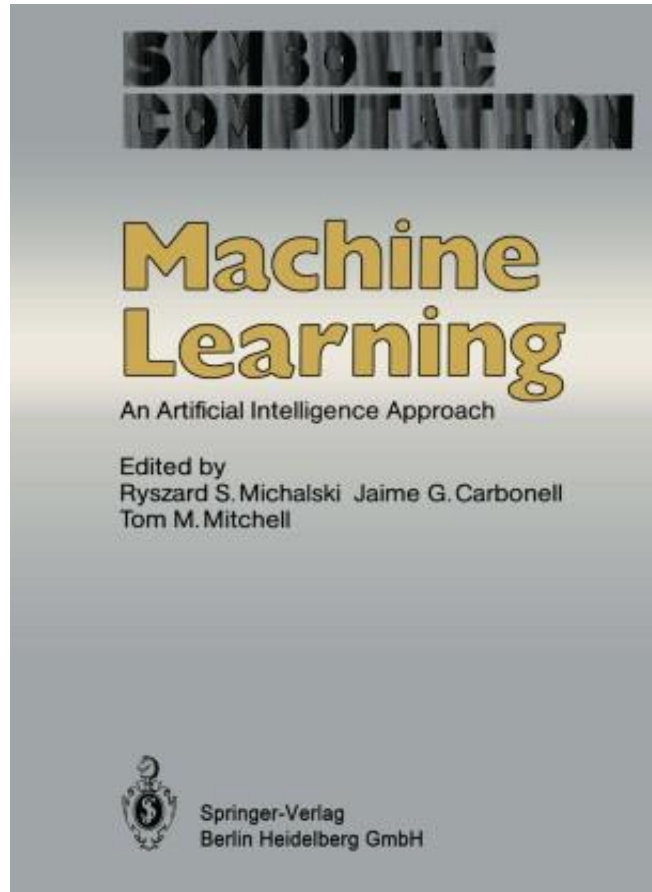


Dennett's minds

O hype sobre sistemas especialistas e IA nos 1980s



IA no final dos anos 1980



Machine Learning established



Brook's subsumption architecture



Hilton's back-propagation

IA durante a década de 1990

Support-Vector Networks

CORINNA CORTES
VLADIMIR VAPNIK
AT&T Bell Labs, Holmdel, NJ 07733, USA

corinna@neural.att.com
vlad@neural.att.com

Editor: Lorenza Saitta

Abstract. The support-vector network is a new learning machine for two-group classification problems. The machine conceptually implements the following idea: input vectors are non-linearly mapped to a very high-dimension feature space. In this feature space a linear decision surface is constructed. Special properties of the decision surface ensures high generalization ability of the learning machine. The idea behind the support-vector network was previously implemented for the restricted case where the training data can be separated without errors. We here extend this result to non-separable training data.

High generalization ability of support-vector networks utilizing polynomial input transformations is demonstrated. We also compare the performance of the support-vector network to various classical learning algorithms that all took part in a benchmark study of Optical Character Recognition.

Keywords: pattern recognition, efficient learning algorithms, neural networks, radial basis function classifiers, polynomial classifiers.

1. Introduction

More than 60 years ago R.A. Fisher (Fisher, 1936) suggested the first algorithm for pattern recognition. He considered a model of two normal distributed populations, $N(\mathbf{m}_1, \Sigma_1)$ and $N(\mathbf{m}_2, \Sigma_2)$ of n dimensional vectors $\mathbf{x}$ with mean vectors $\mathbf{m}_1$ and $\mathbf{m}_2$ and co-variance matrices Σ_1 and Σ_2 , and showed that the optimal (Bayesian) solution is a quadratic decision function:

$$F_{\text{sq}}(\mathbf{x}) = \text{sign} \left[\frac{1}{2} (\mathbf{x} - \mathbf{m}_1)^T \Sigma_1^{-1} (\mathbf{x} - \mathbf{m}_1) - \frac{1}{2} (\mathbf{x} - \mathbf{m}_2)^T \Sigma_2^{-1} (\mathbf{x} - \mathbf{m}_2) + \ln \frac{|\Sigma_2|}{|\Sigma_1|} \right]. \quad (1)$$

In the case where $\Sigma_1 = \Sigma_2 = \Sigma$ the quadratic decision function (1) degenerates to a linear function:

$$F_{\text{lin}}(\mathbf{x}) = \text{sign} \left[(\mathbf{m}_1 - \mathbf{m}_2)^T \Sigma^{-1} \mathbf{x} - \frac{1}{2} (\mathbf{m}_1^T \Sigma^{-1} \mathbf{m}_1 - \mathbf{m}_2^T \Sigma^{-1} \mathbf{m}_2) \right]. \quad (2)$$

To estimate the quadratic decision function one has to determine $\frac{6(n+3)}{2}$ free parameters. To estimate the linear function only n free parameters have to be determined. In the case where the number of observations is small (say less than $10n^2$) estimating $\alpha(n^2)$ parameters is not reliable. Fisher therefore recommended, even in the case of $\Sigma_1 \neq \Sigma_2$, to use the linear discriminator function (2) with Σ of the form:

$$\Sigma = \tau \Sigma_1 + (1 - \tau) \Sigma_2, \quad (3)$$

where τ is some constant¹. Fisher also recommended a linear decision function for the case where the two distributions are not normal. Algorithms for pattern recognition

Vapnik's support vector machines

Reinforcement Learning With Hidden States

Long-Ji Lin
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
e-mail: ljli@cs.cmu.edu

Tom M. Mitchell
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
e-mail: mitchell@cs.cmu.edu

Abstract

Reinforcement learning is an unsupervised learning technique for sequential decision making. *Q-learning* is a widely-used reinforcement learning method. The convergence of Q-learning relies on the Markovian environment assumption, meaning that any information needed to determine the optimal action is reflected in the agent's state representation. If some important state features are missing (or hidden) from the state representation, the true world states cannot be directly identified and optimal decisions cannot be made based on this state representation. This problem is known as the *hidden state problem*. A possible solution to the problem is to use history information to help uncover the hidden features. This paper studies three reinforcement learning architectures that learn to use history to handle hidden states: *windest-Q*, *recurrent-Q*, and *recurrent-model*. Empirical study of these architectures is presented. Their relative strengths and weaknesses are also discussed.

1 Introduction

In the reinforcement learning paradigm, a learning agent continually receives inputs from its sensors, determines its action based on the inputs and its control policy, executes that action, and receives a scalar *reinforcement* or *payoff*. The goal of the agent is to construct an optimal policy so as to maximize its performance, which is often measured by the *discounted cumulative reinforcement* in the future or, for short, *utility*:

$$V_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (1)$$

where V_t is the discounted cumulative reinforcement starting from time t throughout the future, r_t is the

reinforcement received after the transition from time t to $t+1$, and $0 \leq \gamma \leq 1$ is a *discount factor*, which adjusts the importance of long-term consequences of actions.

Q-learning is a widely-used reinforcement learning method [20, 10, 9]. The idea of Q-learning is to construct an evaluation function called *Q-function*:

$$Q(\text{state}, \text{action}) \rightarrow \text{utility} \quad (2)$$

The Q-function is used to predict the discounted cumulative reinforcement (i.e., utility) for each state-action pair given that the agent is in that state and executes that action. Given an optimal Q-function and a state z , the optimal control policy is simply to choose the action a for which $Q(z, a)$ is maximal over all actions.

The optimal Q-function can be incrementally constructed based on *temporal difference* (TD) methods [17], which are a type of dynamic programming [4]. For deterministic domains¹, the utility of an action a in response to a state z is equal to the immediate payoff r plus the best utility that can be obtained from the next state y , discounted by γ . Therefore, the optimal Q-function must satisfy Equation (3):

$$Q(z, a) = r + \gamma \cdot \max_{k \in \text{actions}} [Q(y, k)] \quad (3)$$

During learning, the equation may not hold true, and the difference between the two sides of the equation is what to be minimized. The Q-function thus can be updated in the following way:

$$Q(z, a) \leftarrow Q(z, a) + \eta (r + \gamma \cdot \max_{k \in \text{actions}} [Q(y, k)] - Q(z, a)) \quad (4)$$

where η is the learning rate. The update rule used in this work was in fact more sophisticated and efficient than (4). For further details, see [11, 9].

Q-learning converges and finds the optimal policy under two primary conditions (along with a few weak ones): the Q-function is represented by a lookup table, and the environment is Markovian [21]. The latter means that

¹It is straightforward to extend the discussion here to handle nondeterministic worlds [20, 4].

Copyrighted material

Face Recognition Using Eigenfaces

Matthew A. Turk and Alex P. Pentland
Vision and Modeling Group, The Media Laboratory
Massachusetts Institute of Technology

Abstract

We present an approach to the detection and identification of human faces and describe a working, near-real-time face recognition system which tracks a subject's head and then recognizes the person by comparing characteristics of the face to those of known individuals. Our approach treats face recognition as a two-dimensional recognition problem, taking advantage of the fact that faces are normally upright and thus may be described by a small set of 2-D characteristic views. Face images are projected onto a feature space ("face space") that best encodes the variation among known face images. The face space is defined by the "eigenfaces", which are the eigenvectors of the set of faces; they do not necessarily correspond to isolated features such as eyes, ears, and noses. The framework provides the ability to learn to recognize new faces in an unsupervised manner.

1 Introduction

Developing a computational model of face recognition is quite difficult, because faces are complex, multidimensional, and nonrigid visual stimuli. They are a natural class of objects, and stand in stark contrast to line-wave gratings, the "blocks world", and other artificial stimuli used in human and computer vision research[1]. This article most only visual functions, for which we may construct detailed models of retinal or striate activity; face recognition is a very high level task for which computational approaches can currently only suggest broad constraints on the corresponding neural activity.

We therefore focused our research towards developing a set of early, preattentive pattern recognition capability that does not depend upon having full three-dimensional models or detailed geometry. Our aim was to develop a computational model of face recognition which is fast, reasonably simple, and accurate in constrained environments such as an office or a household.

Although face recognition is a high level visual problem, there is quite a bit of structure imposed on the task. We take advantage of some of this structure by proposing a scheme for recognition which is based on an information theory approach, seeking to encode the most relevant information in a group of faces which will best distinguish them from one

another. The approach transforms face images into a small set of characteristic feature images, called "eigenfaces", which are the principal components of the initial training set of face images. Recognition is performed by projecting a new image into the subspace spanned by the eigenfaces ("face space") and then classifying the face by comparing its position in face space with the positions of known individuals. Automatically learning and later recognizing new faces is practical within this framework. Recognition under reasonably varying conditions is achieved by training on a limited number of characteristic views (e.g., a "straight on" view, a 45° view, and a profile view). The approach has advantages over other face recognition schemes in the speed and simplicity, learning capacity, and relative insensitivity to small or gradual changes in the face image.

1.1 Background and related work

Much of the work in computer recognition of faces has focused on detecting individual features such as the eyes, nose, mouth, and head outline, and defining a face model by the position, size, and relationships among these features. Beginning with Blobb's [2] and Kanade's [3] early systems, a number of automated or semi-automated face recognition strategies have modeled and classified faces based on normalized distances and ratios among feature points. Recently this general approach has been continued and improved by the recent work of Vallo et al. [4].

Such approaches have proven difficult to extend to multiple views, and have often been quite fragile. Research in human strategies of face recognition, moreover, has shown that individual features and their immediate relationships comprise an inefficient representation to account for the performance of adult human face identification [5]. Nonetheless, this approach to face recognition remains the most popular one in the computer vision literature.

Connectionist approaches to face identification seek to capture the configurational, or gestalt-like nature of the task. Fleming and Cottrell [6], building on earlier work by Kolosn and Lathin [7], use nonlinear units to train a network via back propagation to classify face images. Stankovic's WISARD system [8] has been applied with some success to large face images, recognizing both identity and expression. Most connectionist systems dealing with faces treat the input image as a general 2-D pattern.

CH2983-5/91/0000-0586/91.00 © 1991 IEEE

586

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on November 25, 2009 at 08:08 from IEEE Xplore. Restrictions apply.

Pentland's Eigenfaces

IA durante os anos 2000

Stanley: The Robot that Won the DARPA Grand Challenge

Sebastian Thrun, Mike Montemerlo, Hendrik Dahlkamp, David Stavens, Andrei Aron, James Diebel, Philip Fong, John Gale, Morgan Halpern, Gabriel Hoffmann, Kenny Lau, Celia Oakley, Mark Palatucci, Vaughan Pratt, and Pascal Stang
Stanford Artificial Intelligence Laboratory
Stanford University
Stanford, California 94305

Sven Strohband, Cedric Dupont, Lars-Erik Jendrosseck, Christian Koelen, Charles Markey, Carlo Rummel, Joe van Niekerk, Eric Jensen, and Philippe Alessandrini
Volkswagen of America, Inc.
Electronics Research Laboratory
4009 Miranda Avenue, Suite 100
Palo Alto, California 94304

Gary Bradski, Bob Davies, Scott Ettinger, Adrian Kaehler, and Ara Nefian
Intel Research
2200 Mission College Boulevard
Santa Clara, California 95052

Pamela Mahoney
Mohr Davidow Ventures
3000 Sand Hill Road, Bldg. 3, Suite 290
Menlo Park, California 94025

Received 13 April 2006; accepted 17 June 2006

Journal of Field Robotics 23(9), 661–699 (2006) © 2006 Wiley Periodicals, Inc.
Published online in Wiley InterScience (www.interscience.wiley.com). • DOI: 10.1002/rob.20147



Stanley

Natural Language Engineering 10 (3/4): 327–348. © 2004 Cambridge University Press
DOI: 10.1017/S1351324904000323 Printed in the United Kingdom

UIMA: an architectural approach to unstructured information processing in the corporate research environment

DAVID FERRUCCI and ADAM LALLY

IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA
e-mail: {ferrucci, alally}@us.ibm.com

(Received 1 July 2003; revised 9 February 2004)

Abstract

IBM Research has over 200 people working on Unstructured Information Management (UIM) technologies with a strong focus on Natural Language Processing (NLP). These researchers are engaged in activities ranging from natural language dialog, information retrieval, topic-tracking, named-entity detection, document classification and machine translation to bioinformatics and open-domain question answering. An analysis of these activities strongly suggested that improving the organization's ability to quickly discover each other's results and rapidly combine different technologies and approaches would accelerate scientific advance. Furthermore, the ability to reuse and combine results through a common architecture and a robust software framework would accelerate the transfer of research results in NLP into IBM's product platforms. Market analyses indicating a growing need to process unstructured information, specifically multilingual, natural language text, coupled with IBM Research's investment in NLP, led to the development of middleware architecture for processing unstructured information dubbed UIMA. At the heart of UIMA are powerful search capabilities and a data-driven framework for the development, composition and distributed deployment of analysis engines. In this paper we give a general introduction to UIMA focusing on the design points of its analysis engine architecture and we discuss how UIMA is helping to accelerate research and technology transfer.

1 Motivation and objectives

We characterize a UIM application as a software system that analyzes large volumes of unstructured information while consulting structured information sources to discover, organize and present knowledge relevant to the application's end-user. We define *structured information* as information whose intended meaning is unambiguous and explicitly represented in the structure or format of the data. The canonical example of structured information is a relational database table. We define *unstructured information* as information whose intended meaning is only loosely implied by its form. The canonical example is a natural language document; other examples include voice, audio, image and video.

Market indicators suggest a rapid increase in the use of text and voice analytics to a growing class of commercially viable applications (Roush 2003). National security

UIMA

Real-Time Human Pose Recognition in Parts from Single Depth Images

Jamie Shotton Andrew Fitzgibbon Mat Cook Toby Sharp Mark Finocchio
Richard Moore Alex Kipman Andrew Blake
Microsoft Research Cambridge & Xbox Incubation

Abstract

We propose a new method to quickly and accurately predict 3D positions of body joints from a single depth image, using no temporal information. We take an object recognition approach, designing an intermediate body parts representation that maps the difficult pose estimation problem into a simpler per-pixel classification problem. Our large and highly varied training dataset allows the classifier to estimate body parts invariant to pose, body shape, clothing, etc. Finally we generate confidence-scored 3D proposals of several body joints by projecting the classification result and finding local modes.

The system runs at 200 frames per second on consumer hardware. Our evaluation shows high accuracy on both synthetic and real test sets, and investigates the effect of several training parameters. We achieve state of the art accuracy in our comparison with related work and demonstrate improved generalization over exact whole-skeleton nearest neighbor matching.

1. Introduction

Robust interactive human body tracking has applications including gaming, human-computer interaction, security, telepresence, and even health-care. The task has recently been greatly simplified by the introduction of real-time depth cameras [16, 19, 44, 37, 28, 13]. However, even the best existing systems still exhibit limitations. In particular, until the launch of Kinect [21], none ran at interactive rates on consumer hardware while handling a full range of human body shapes and sizes undergoing general body motions. Some systems achieve high speeds by tracking from frame to frame but struggle to re-initialize quickly and so are not robust. In this paper, we focus on pose recognition in parts: detecting from a single depth image a small set of 3D position candidates for each skeletal joint. Our focus on per-frame initialization and recovery is designed to complement any appropriate tracking algorithm [7, 39, 16, 42, 13] that might further incorporate temporal and kinematic coherence. The algorithm presented here forms a core component of the Kinect gaming platform [21].

Illustrated in Fig. 1 and inspired by recent object recognition work that divides objects into parts (e.g. [12, 43]), our approach is driven by two key design goals: computational efficiency and robustness. A single input depth image is segmented into a dense probabilistic body part labeling, with the parts defined to be spatially localized near skeletal

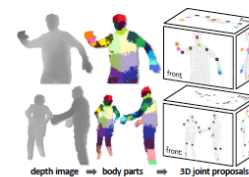


Figure 1. Overview. From a single input depth image, a per-pixel body part distribution is inferred. (Colors indicate the most likely part labels at each pixel, and correspond in the joint proposals). Local modes of this signal are estimated to give high-quality proposals for the 3D locations of body joints, even for multiple users.

joints of interest. Reprojecting the inferred parts into world space, we localize spatial modes of each part distribution and thus generate (possibly several) confidence-weighted proposals for the 3D locations of each skeletal joint.

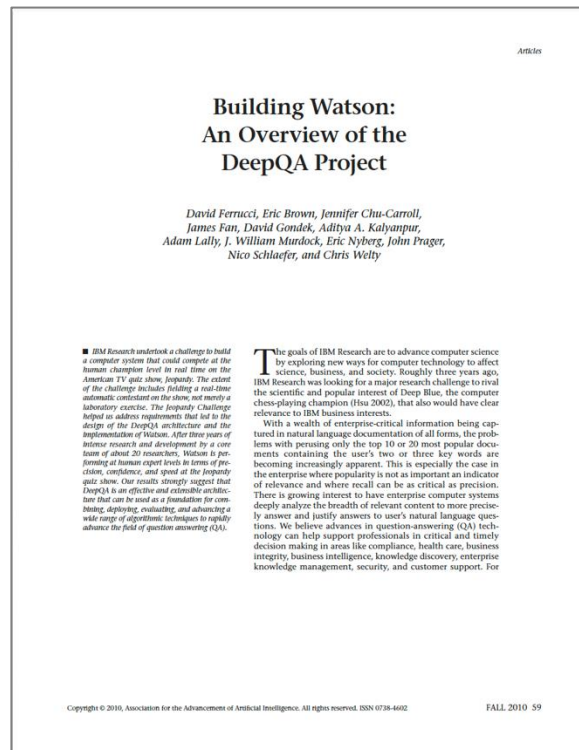
We treat the segmentation into body parts as a per-pixel classification task (no pairwise terms or CRF have proved necessary). Evaluating each pixel separately avoids a combinatorial search over the different body joints, although within a single part there are of course still dramatic differences in the contextual appearance. For training data, we generate realistic synthetic depth images of humans of many shapes and sizes in highly varied poses sampled from a large motion capture database. We train a deep randomized decision forest classifier which avoids overfitting by using hundreds of thousands of training images. Simple, discriminative depth comparison image features yield 3D translation invariance while maintaining high computational efficiency. For further speed, the classifier can be run in parallel on each pixel on a GPU [34]. Finally, spatial modes of the inferred per-pixel distributions are computed using mean shift [10] resulting in the 3D joint proposals.

An optimized implementation of our algorithm runs in under 5ms per frame (200 frames per second) on the Xbox 360 GPU, at least one order of magnitude faster than existing approaches. It works frame-by-frame across dramatically differing body shapes and sizes, and the learned discriminative approach naturally handles self-occlusions and

1

Kinect

IBM Watson wins Jeopardy! in 2011



Building Watson



REVIEW

doi:10.1038/nature14839

Deep learning

Yann LeCun^{1,2}, Yoshua Bengio³ & Geoffrey Hinton^{4,5}

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers hierarchical structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representations in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

Machine learning technology powers many aspects of modern society from web searches to content filtering on social networks to recommendations on e-commerce websites, and is increasingly present in consumer products such as cameras and smartphones. Machine learning systems are used to identify objects in images, transcribe speech into text, match news items, point or products with users' interests, and select relevant results of search. Increasingly these applications make use of a class of techniques called deep learning.

Conventional machine-learning techniques were limited in their ability to process natural data in their raw form. For decades, constructing a pattern-recognition or machine-learning system required careful engineering and considerable domain expertise to design a feature extractor that transformed the raw data (such as the pixel values of an image) into a suitable internal representation or feature vector from which the learning algorithm, often a classifier, could detect or classify patterns in the input.

Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification. Deep-learning methods are representation-learning methods with multiple levels of representation, obtained by composing simple but non-linear models that each transform the representation at one level (starting with the raw input) into a representation at a higher, slightly more abstract level. With the composition of enough such transformations, very complex functions can be learned. For classification tasks, higher layers of representation amplify aspects of the input that are important for discrimination and suppress irrelevant variations. An image, for example, comes in the form of an array of pixel values, and the learned features in the first layer of representation typically represent the presence or absence of edges at particular orientations and locations in the image. The second layer typically detects motifs by grouping particular arrangements of edges, regardless of small variations in the edge positions. The third layer may assemble motifs into larger combinations that correspond to parts of familiar objects, and subsequent layers would detect objects as combinations of these parts. The key aspect of deep learning is that these layers of features are not designed by human engineers, but they are learned from data using a general-purpose learning procedure.

Deep learning is making major advances in solving problems that have resisted the best attempts of the artificial intelligence community for many years. It has turned out to be very good at discovering

intricate structure in large, high-dimensional data and is therefore applicable to many domains of science, business and government. In addition to learning records in image recognition¹ and 'speech recognition'² it has been used for machine-learning techniques at predicting the activity of potential drug molecules³, analysing text acceleration data^{4,5}, recommending books to users⁶, and predicting the effect of mutations on gene coding DNA on gene expression and disease^{7,8}. Perhaps most surprisingly, deep learning has produced extremely promising results for various tasks in natural language understanding⁹, particularly speech classification, sentiment analysis, question answering¹⁰ and language translation^{11,12}.

We think that deep learning will have many more successes in the near future because it requires very little engineering by hand, so it can easily take advantage of increases in the amount of available computation and data. New learning algorithms and architectures that are currently being developed for deep neural networks will only accelerate this progress.

Supervised learning
The most common form of machine learning, deep or not, is supervised learning, meaning that we want to build a system that can classify images as containing, say, a house, a car, a person or a pet. We first collect a large data set of images of houses, cars, people and pets, each labelled with its category. During training, the machine is shown an image and produces an output in the form of a vector of scores, one for each category. We want the desired category to have the highest score of all categories, but this is usually only happen before training. We compute an objective function that measures the error (or distance) between the output scores and the desired pattern of scores. The machine then modifies its internal adjustable parameters to reduce this error. These adjustable parameters, often called weights, are real numbers that can be seen as a family that define the input-output function of the machine. In a typical deep-learning system, there may be hundreds of millions of these adjustable weights, and hundreds of millions of labelled examples with which to train the machine.

To properly adjust the weights vector, the learning algorithm computes a gradient vector that, for each weight, indicates by what amount the error would increase or decrease if the weight were increased by any amount. The weight vector is then adjusted in the opposite direction to the gradient vector.

The objective function, averaged over all the training examples, can

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
krizale@utoronto.ca

Ilya Sutskever
University of Toronto
ils@cs.toronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet ILSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-overlapping feature maps, and three fully connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently developed regularization method called 'dropout' that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 error rate of 15.3%, compared to 26.2% achieved by the second best entry.

1 Introduction

Current approaches to object recognition make essential use of machine learning methods. To improve their performance, we can collect larger datasets, learn more powerful models, and use better techniques for preventing overfitting. Until recently, datasets of labeled images were relatively small — on the order of tens of thousands of images (e.g., NORB [16], Caltech-101/256 [8, 9], and CIFAR-10/100 [12]). Simple recognition tasks can be solved quite well with datasets of this size, especially if they are augmented with label-preserving transformations. For example, the current best error rate on the MNIST digit-recognition task (<0.3%) approaches human performance [4]. But objects in realistic settings exhibit considerable variability, so to learn to recognize them it is necessary to use much larger training sets. And indeed, the shortcomings of small image datasets have been widely recognized (e.g., Poggio et al. [21]), but it has only recently become possible to collect labeled datasets with millions of images. The new larger datasets include LabelMe [23], which consists of hundreds of thousands of fully segmented images, and ImageNet [6], which consists of over 15 million labeled high-resolution images in over 22,000 categories.

To learn about thousands of objects from millions of images, we need a model with a large learning capacity. However, the immense complexity of the object recognition task means that this problem cannot be specified even by a dataset as large as ImageNet, so our model should also have lots of prior knowledge to compensate for all the data we don't have. Convolutional neural networks (CNNs) constitute one such class of models [16, 11, 13, 15, 22, 26]. They also can be constructed by varying their depth and breadth, and they also make strong and mostly correct assumptions about the nature of images (namely, stationarity of statistics and locality of pixel dependencies). Thus, compared to standard feedforward neural networks with similarly sized layers, CNNs have much fewer connections and parameters and so they are easier to train, while their theoretical best performance is likely to be only slightly worse.

1

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
mikolov@google.com

Ilya Sutskever
Google Inc.
ils@cs.toronto.ca

Kai Chen
Google Inc.
kai@google.com

Greg Corrado
Google Inc.
gcorrado@google.com

Jeffrey Dean
Google Inc.
jeff@cs.toronto.ca

Abstract

The recently introduced continuous Skip-gram model is an efficient method for learning high-quality distributed vector representations that capture a large number of precise syntactic and semantic word relationships. In this paper we present several extensions that improve both the quality of the vectors and the training speed. By subsampling of the frequent words we obtain significant speedup and also learn more regular word representations. We also describe a simple alternative to the hierarchical softmax called negative sampling. An inherent limitation of word representations is their indifference to word order and their inability to represent idiomatic phrases. For example, the meanings of 'Canada' and 'Air' cannot be easily combined to obtain 'Air Canada'. Motivated by this example, we present a simple method for finding phrases in text, and show that learning good vector representations for millions of phrases is possible.

1 Introduction

Distributed representations of words in a vector space help learning algorithms to achieve better performance in natural language processing tasks by grouping similar words. One of the earliest use of word representations dates back to 1966 by the Elman, Rumelhart, and Williams [13]. This idea has since been applied to statistical language modeling with considerable success [1]. The follow-up work includes applications to automatic speech recognition and machine translation [14, 7], and a wide range of NLP tasks [12, 20, 15, 3, 18, 19, 9].

Recently, Mikolov et al. [8] introduced the Skip-gram model, an efficient method for learning high-quality word representations of words from large amounts of unannotated text data. Unlike most of the previously used neural network architectures for learning word vectors, training of the Skip-gram model (see Figure 1) does not involve dense matrix multiplications. This makes the training extremely efficient, as optimized single-matrix implementation can train on more than 100 billion words in one day.

The word representations computed using neural networks are very interesting because the learned vectors explicitly encode many linguistic regularities and patterns. Surprisingly, surprisingly, many of these patterns can be interpreted as linear transformations. For example, the result of a vector addition $\text{vec}(\text{Master}) - \text{vec}(\text{Queen}) + \text{vec}(\text{France})$ is closer to $\text{vec}(\text{Paris})$ than to any other word vector [8, 9].

1

LETTER

doi:10.1038/nature14838

Human-level control through deep reinforcement learning

Vishakh Mittal¹, Koray Kavukcuoglu², David Silver³, Andrej A. Ruess⁴, Joel Veness⁵, Marc G. Bellemare⁶, Alex Graves⁷, Martin Riedmiller⁸, Andreas B. Pfaffel⁹, George Ostrovski¹⁰, Greg Deacon¹¹, Charles Beattie¹², Amir Shalek¹³, Laurent Breilinger¹⁴, Helen King¹⁵, Dhanraj Kannan¹⁶, Dan Wierstra¹⁷, Shane Legg¹⁸ & Demis Hassabis¹⁹

The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

Reinforcement learning is known to be suitable for even to design agents that learn to control a wide range of systems, from simple robots to complex systems. The theory of reinforcement learning provides a normative account, deeply rooted in psychological and neuroscientific perspectives on animal behaviour, of how agents may optimally control their actions in an environment. Tens of thousands of algorithms have been devised for formalizing this quest for artificial general intelligence, but progress has been limited by a lack of a unifying theoretical framework. Here, we introduce a family of deep reinforcement learning algorithms that combine the benefits of deep learning with the power of Monte Carlo methods to provide a general framework for learning from sparse rewards. Our algorithms achieve state-of-the-art performance on a wide range of tasks, including Atari 2600 games, and demonstrate human-level performance on a broad range of tasks, including the game of Go.

©2015 Macmillan Publishers Limited. All rights reserved.

Generative Adversarial Nets

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio¹
Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimum two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to 1 everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

1 Introduction

The promise of deep learning is to discover rich, hierarchical models [2] that represent probability distributions over the kinds of data encountered in artificial intelligence applications, such as natural images, audio waveforms containing speech, and symbols in natural language corpora. So far, the most striking successes in deep learning have involved discriminative models, usually those that map high-dimensional, rich sensory input to a class label [14, 22]. These striking successes have primarily been based on the backpropagation and dropout algorithms, using receptive linear units [19, 9, 16] which have a particularly well-behaved gradient. Deep generative models have had less of an impact, due to the difficulty of approximating many intractable probabilistic computations that arise in maximum likelihood estimation and related strategies, and due to difficulty of leveraging the benefits of piecewise linear units in the generative context. We propose a new generative model estimation procedure that sidesteps these difficulties.¹

In the proposed adversarial net framework, the generative model is pitted against an adversary, a discriminative model that learns to discriminate whether a sample is from the model distribution or the data distribution. The generative model can be thought of as analogous to a team of counterfeiter, trying to produce fake currency and use it without detection, while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeiters are indistinguishable from the genuine articles.

¹ Jean Pouget-Abadie is a visiting University of Montreal from Ecole Polytechnique.
² Sherjil Ozair is a visiting University of Montreal from Indian Institute of Technology Delhi.
³ Yoshua Bengio is a CIFAR Senior Fellow.
All code and hyperparameters available at <http://www.github.com/goodfellow/adversarial>.

1

Deep Learning

Convolutional Networks

Word2vec

Deep Reinforcement Learning

GANs

Resumo da Inteligência Artificial (até 2017)

1950

Teste de Turing

Alan Turing propôs o Jogo da Imitação como forma de avaliar inteligência de máquinas.

1956

IA Nasce

John McCarthy criou o termo "Inteligência Artificial" durante conferência em Dartmouth com todos os pioneiros de IA.

1965

Eliza

Joseph Weizenbaum programou Eliza, um sistema que é capaz de simular o processo de conversação com humanos.

1975

MYCIN

Edward Feigenbaum construiu MYCIN, um sistema especialista baseado em regras tão bom quanto humanos na identificação de bactérias causadoras de infecções e na recomendação de antibióticos.

1986

Redes Neurais

Geoffrey Hinton propôs o algoritmo de "Back Propagation" que permite que redes neurais aprendam a partir de dados.

1986

Robôs Inteligentes

Rod Brooks propôs a programação de robôs usando a "Subsumption Architecture" que possibilita o uso de robôs inteligentes no mundo real.

1997

Deep Blue

O IBM Deep Blue tornou-se o primeiro computador a vencer um campeão mundial de xadrez, Garry Kasparov. Com 32 processadores, a máquina calculava cerca de 200 milhões de jogadas de xadrez por segundo.

1998

Aprendizagem de Máquina

Michael I. Jordan propôs o uso de probabilidades na forma de "Graphical Models" que estabelece os princípios fundamentais do "Aprendizado de Máquina" moderno.

2009

ImageNet

Fei-Fei Li publicou o ImageNet, um conjunto de mais de 10 milhões de imagens rotuladas, fundamental para o desenvolvimento das redes neurais nos últimos 10 anos.

2011

Watson

O Watson, inteligência artificial da IBM, tornou-se conhecido após derrotar os dois maiores campeões do jogo de perguntas e respostas Jeopardy, inaugurando o uso de big data em IA.

2012

Deep Learning

Andrew Ng mostrou que GPUs (um tipo de chip usado para jogos de computador) podem acelerar em 100x o treinamento de redes neurais, permitindo o seu uso em problemas reais.

2017

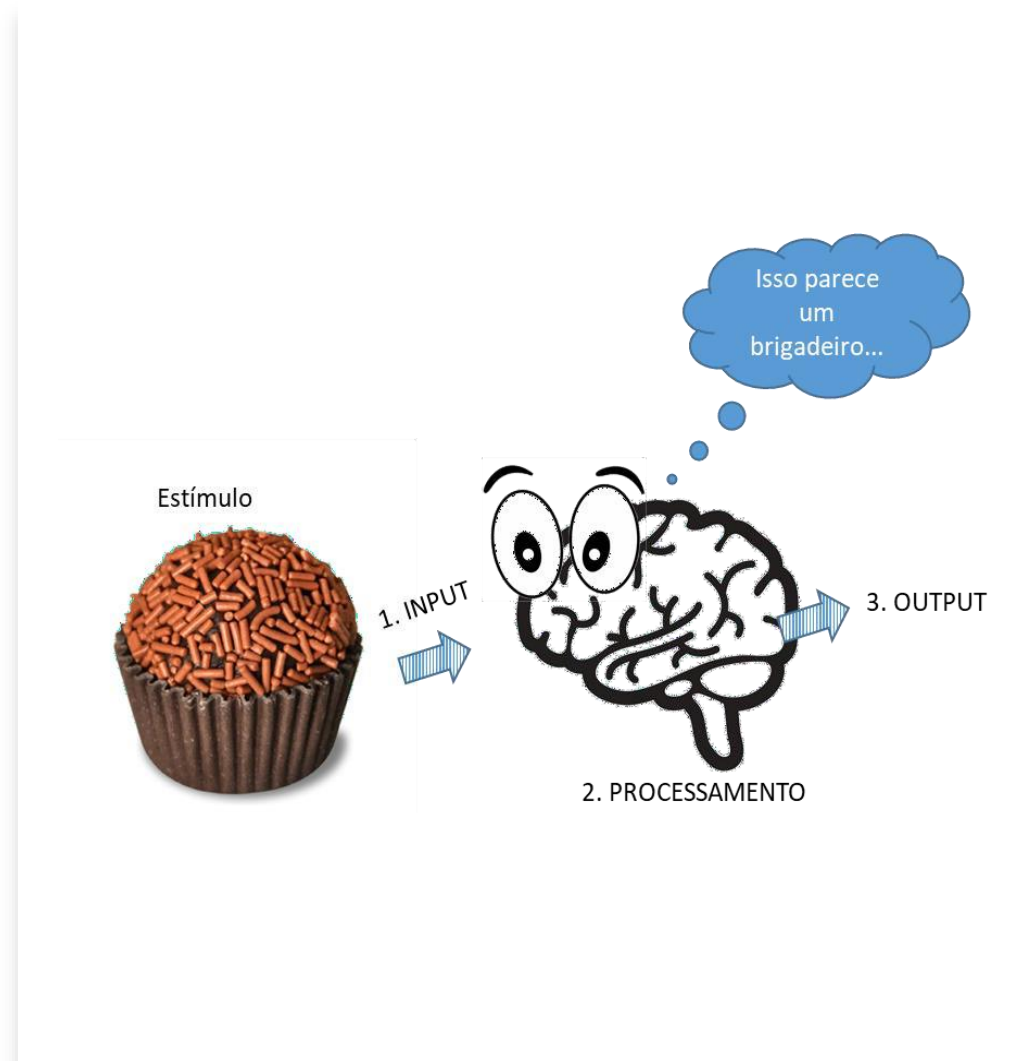
AlphaGo

O programa AlphaGo da DeepMind derrotou o campeão Lee Sedol em uma competição de Go, jogo considerado muito mais difícil para máquinas que o xadrez.

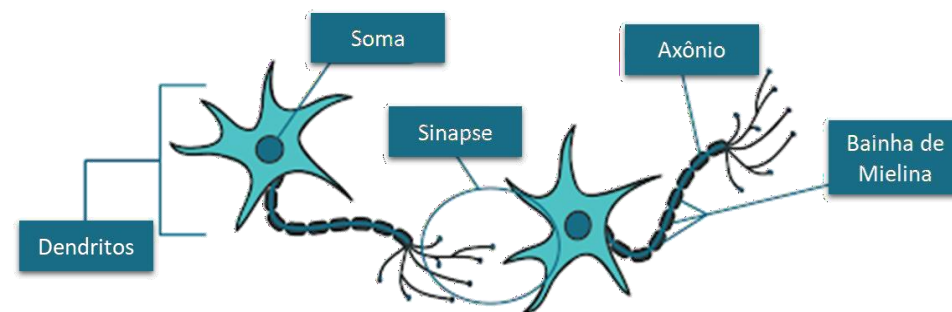
O que faltou nessa lista?

Redes Neurais Artificiais

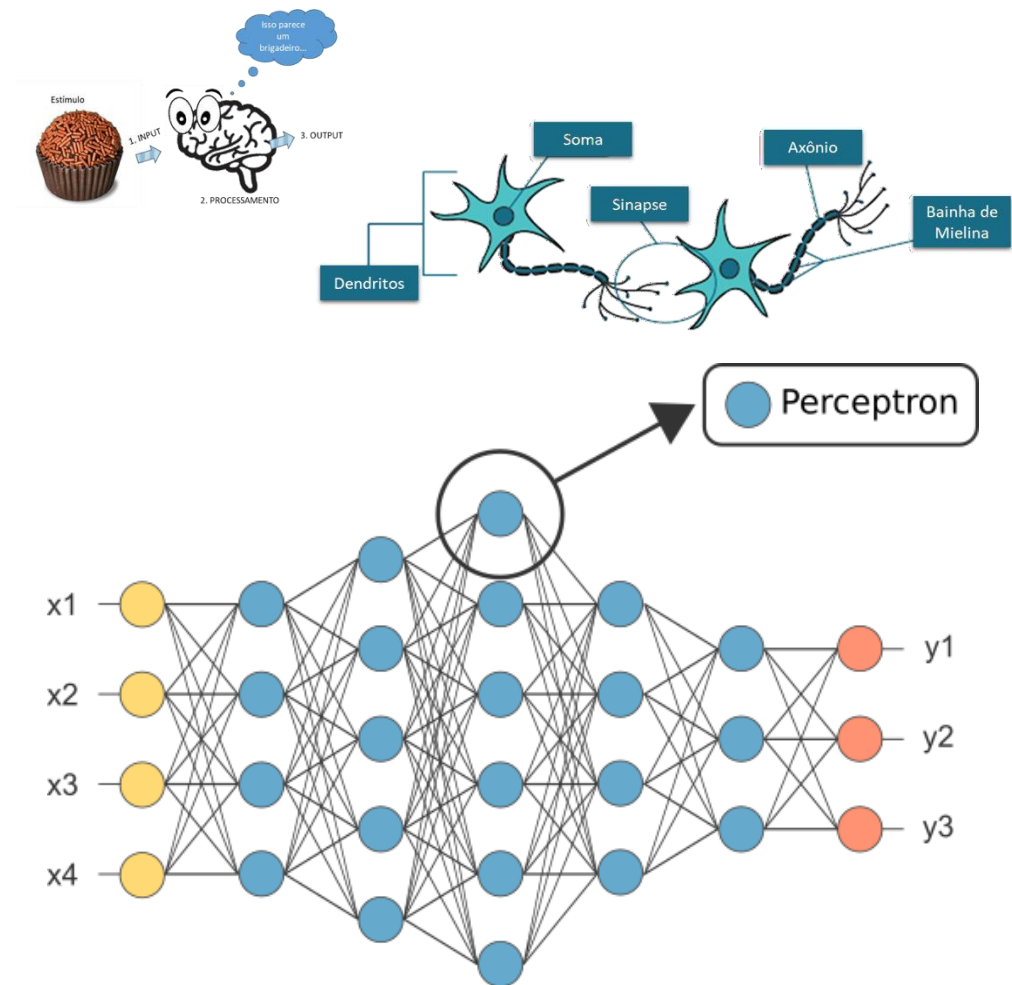
Conceito de Redes Neurais



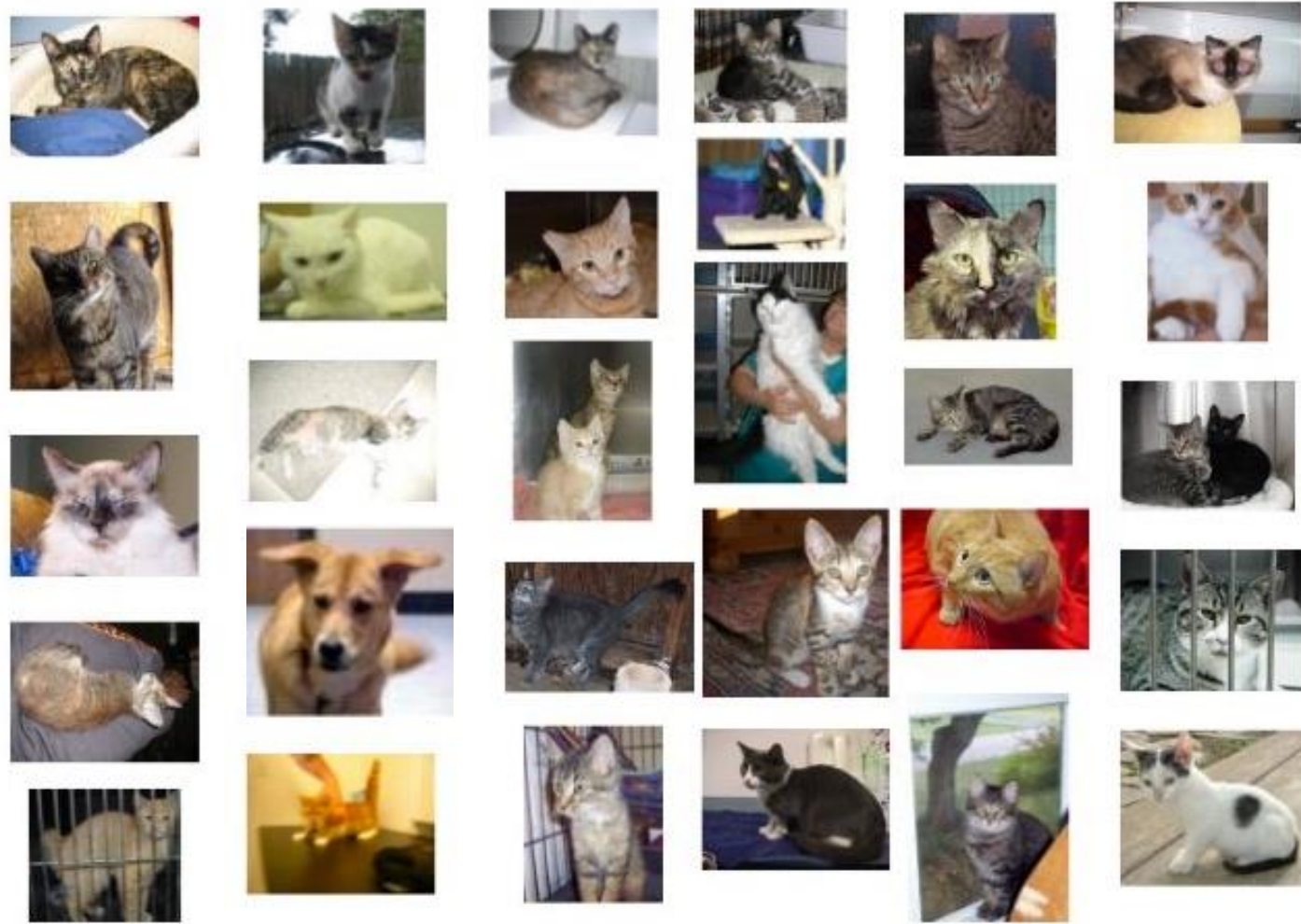
Conceito de Redes Neurais



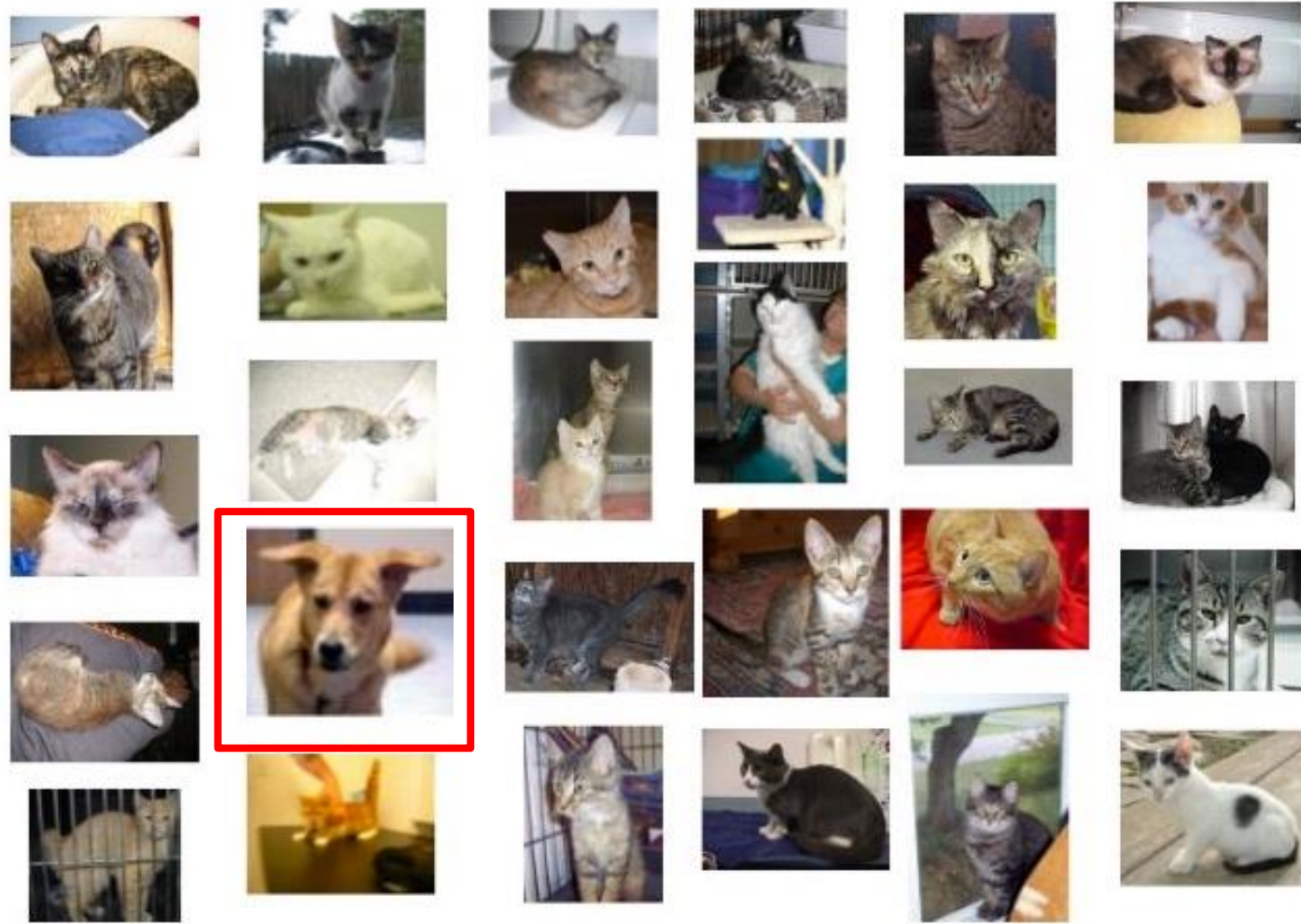
Conceito de Redes Neurais



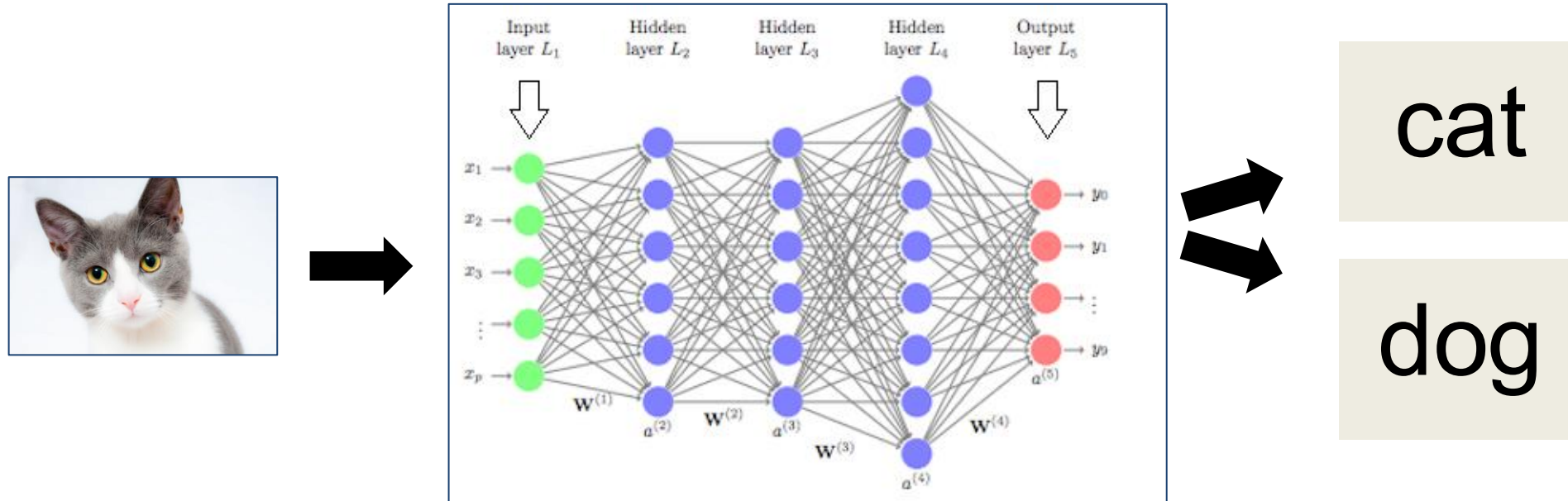
Qual animal não é um gato?



Qual animal não é um gato?



Redes Neurais são funções matemáticas complexas



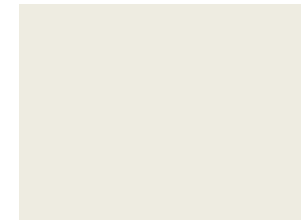
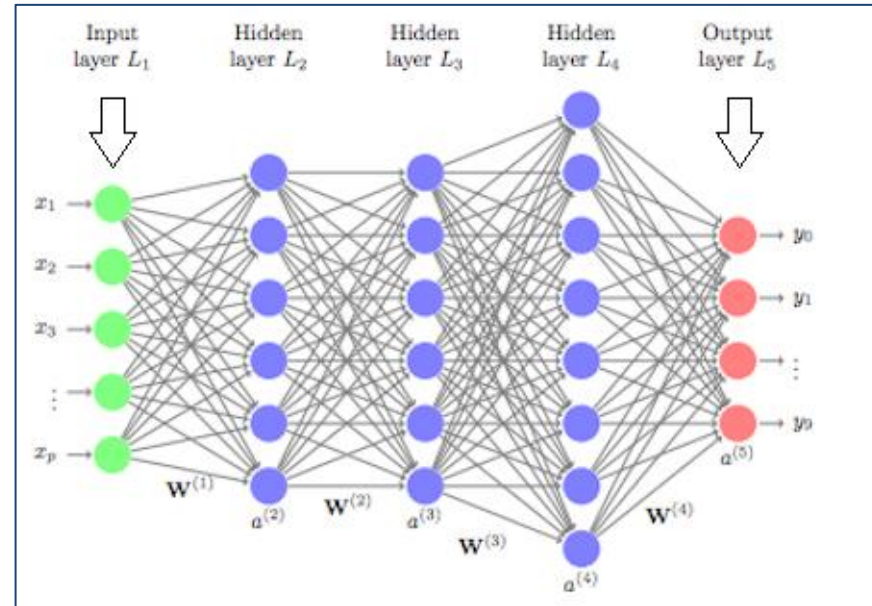
função matemática (muitos pesos)

RNAs são treinadas iterativamente com exemplos

Exemplo:



cat



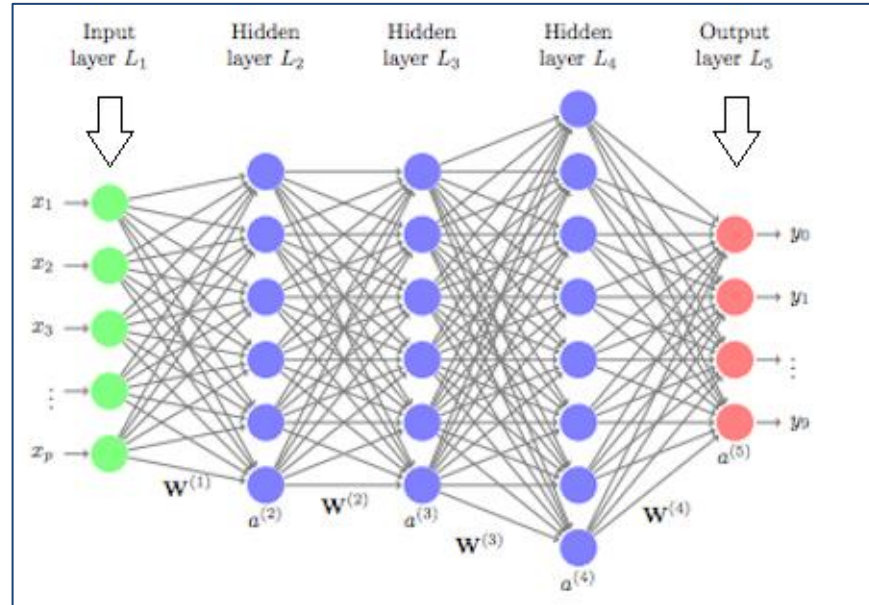
cálculo do exemplo

RNAs são treinadas iterativamente com exemplos

Exemplo:



cat



dog

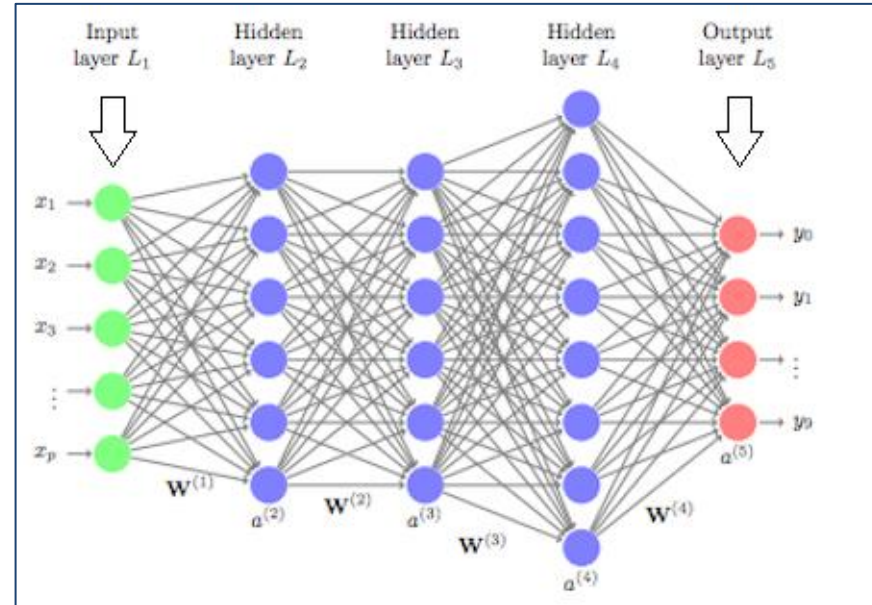
cálculo do exemplo

Os exemplos são usados para ajustar os pesos

Example:



cat



dog



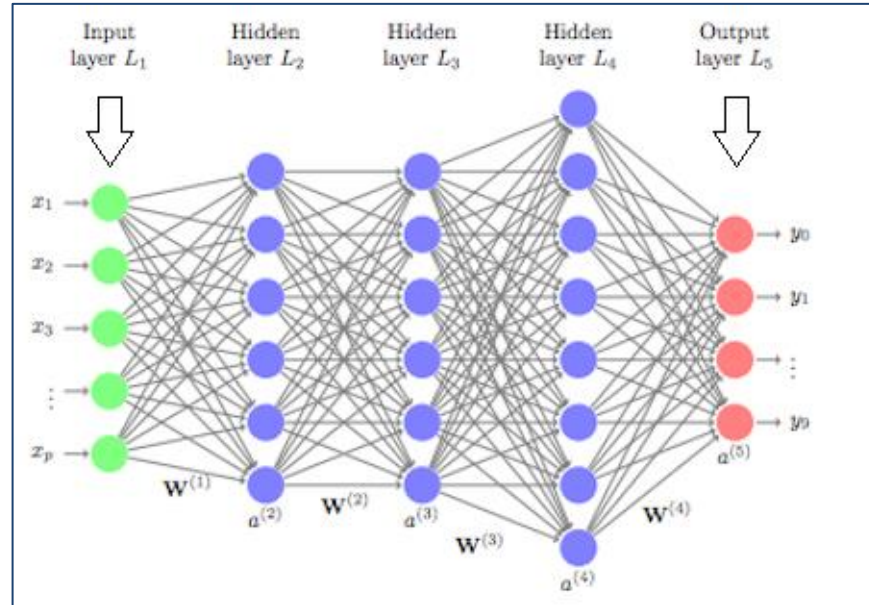
back propagation (ajustando pesos)

Para que, da próxima vez, a resposta seja correta

Example:



cat



cat



recálculo do exemplo

Histórico



~1950 – Primeiros conceitos de redes neurais

Não havia uma forma eficiente de se treinar



1980 – Pesquisas com backpropagation

Essencial para o treinamento eficiente



1990 – Primeiro uso no mundo real

Leitura de códigos postais

Chollet, F. Deep Learning with Python, Manning Shelter Island Editor, 2018.

Histórico

- 2010 – Uso em classificação de imagens
- Desafio ImageNet – 1.4 milhão de imagens em 1000 categorias

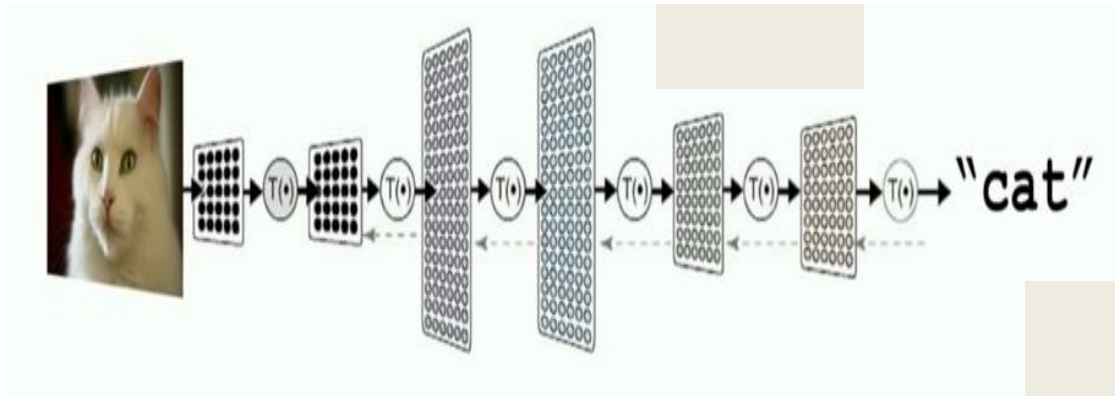


- 2011 – Métodos “clássicos” → 74.3% de acerto (top-five)
- 2012 – RNA → 83.6% de acerto
- 2015 – RNA → 96.4% de acerto → Problema considerado resolvido

<https://image-net.org/>

Chollet, F. Deep Learning with Python, Manning Shelter Island Editor, 2018.

Deep Learning: redes neurais com múltiplas camadas + cálculo rápido usando GPUs



Deep Neural Networks



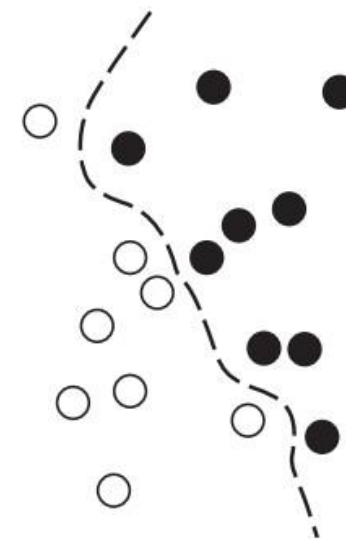
Graphical Processing Units (GPUs)

Histórico de Limitações

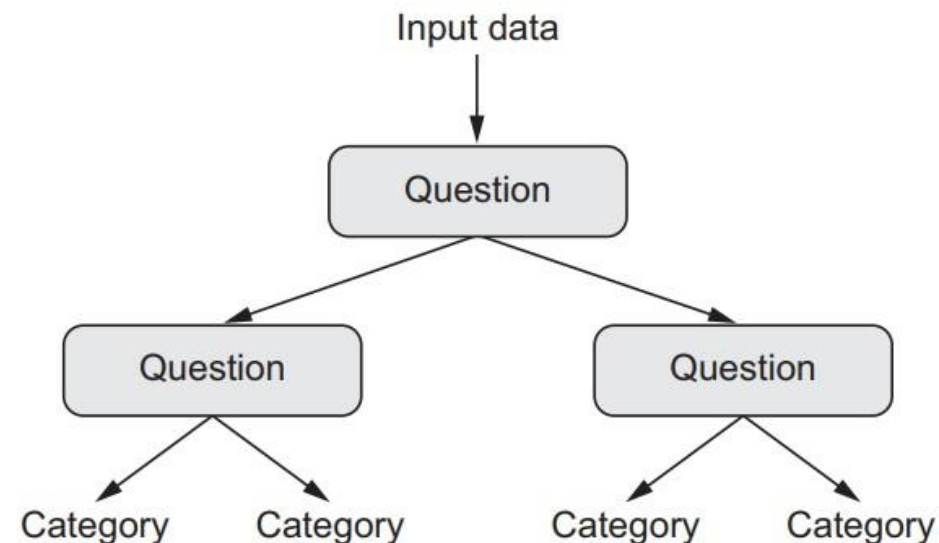
- 1968 – Limites intrínsecos por Minsky and Papert 1968
- 1990-2000 – “Superadas” por SVM
- 2000-2010 – “Superadas” por *Random Forests*

[Minsky, M. and Papert, S., 1969. An introduction to computational geometry. Cambridge tiass., HIT, 479(480)]

[Chollet, F. Deep Learning with Python, Manning Shelter Island Editor, 2018]



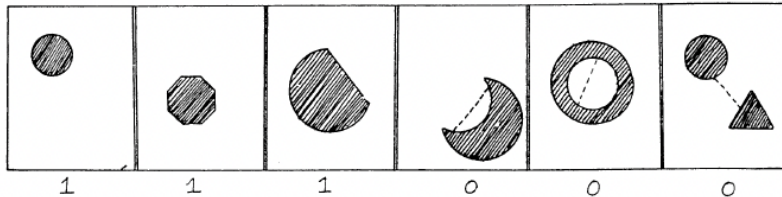
SVM



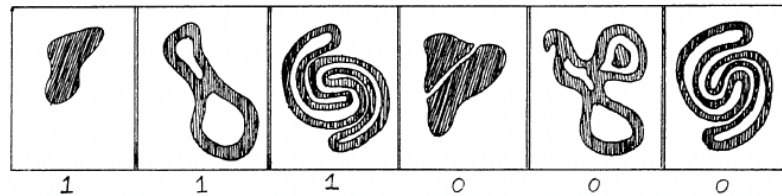
Decision tree → Random Forests

Limites de Perceptrons

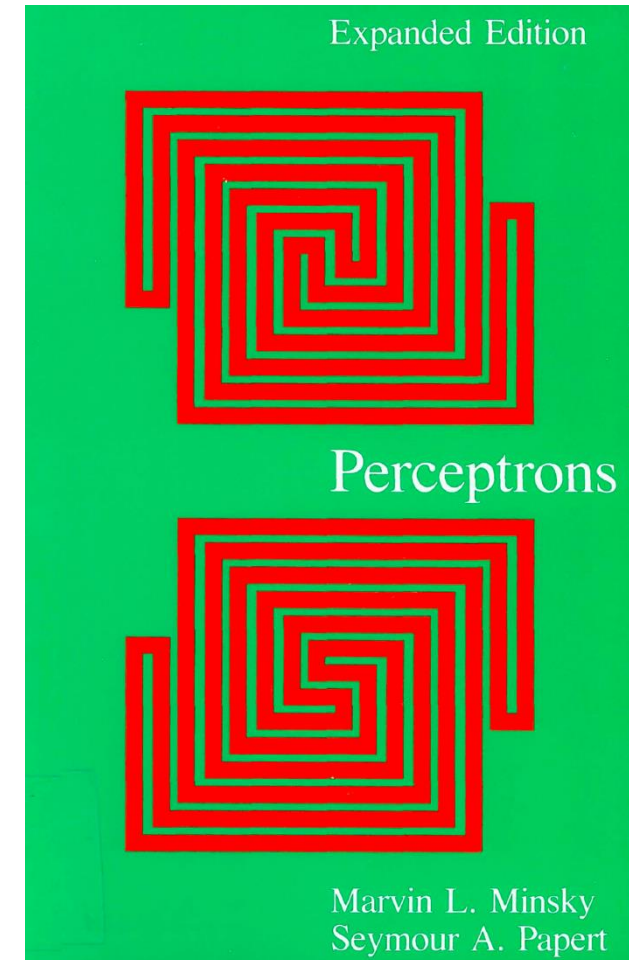
$$\psi_{\text{CONVEX}}(X) = \begin{cases} 1 & \text{if } X \text{ is a convex figure,} \\ 0 & \text{if } X \text{ is not a convex figure;} \end{cases}$$



$$\psi_{\text{CONNECTED}}(X) = \begin{cases} 1 & \text{if } X \text{ is a connected figure,} \\ 0 & \text{otherwise.} \end{cases}$$



[Minsky, M. and Papert, S., 1969. An introduction to computational geometry. Cambridge tiass., HIT, 479(480)]



Estrutura de uma RNA

O que é uma RNA?

- Uma grande função
- Mapeia um conjunto de dados de entrada $x_i \in X$ para o valor alvo desejado $y_i \in Y$.
- Entrada definida por uma matriz $X \in \mathbb{R}^{n_x, m}$
 - n_x é o número de *features* do problema (número de entradas de cada exemplo)
 - m é o número de dados disponíveis (número total de exemplos de treinamento)
- Saída definida por $Y \in \mathbb{R}^{n_y, m}$
 - n_y é o número de saídas da RNA

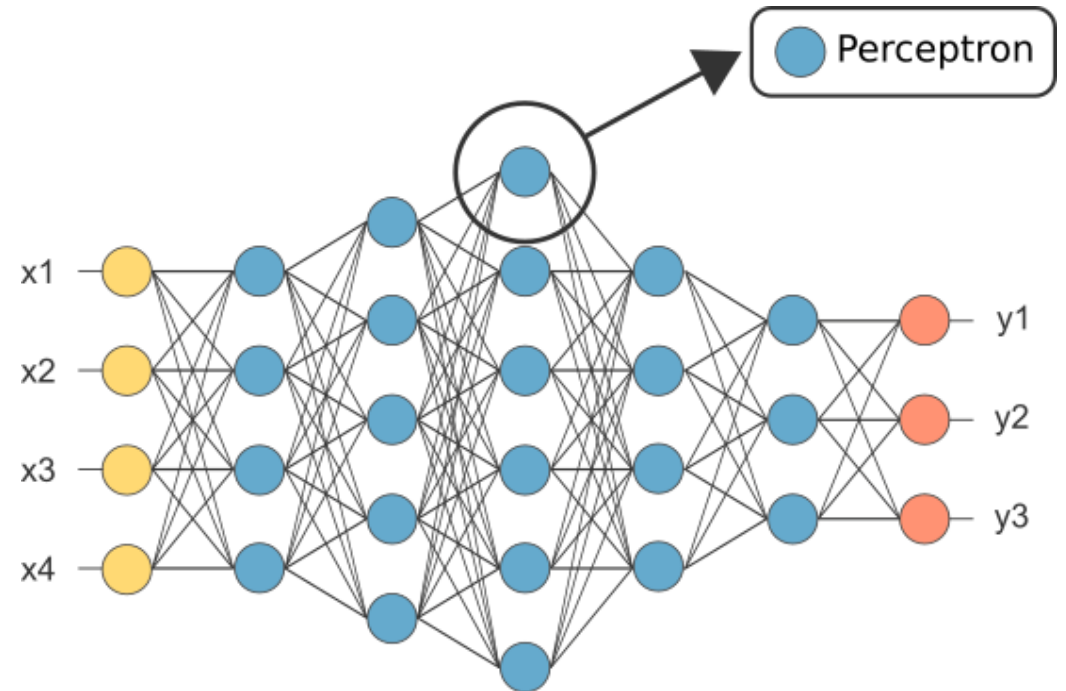


Imagem retirada de:

<https://www.oficinadanet.com.br/tecnologia/25007-o-que-sao-as-redes-neurais-artificiais>

Principais características das redes neurais biológicas estão presentes nas RNAs

- Uma RNA consiste de uma grande quantidade de unidades de processamento simples (neurônios) interconectados;
- Cada neurônio artificial recebe muitos sinais;
- Os sinais recebidos pelos neurônios são modificados por um peso nas sinapses receptoras;
- Os neurônios artificiais somam de forma ponderada as entradas;
- Os neurônios definem a importância da informação e transmitem uma única saída;
- A saída de um neurônio é transmitida para muitos outros neurônios;
- Uma rede pode possuir várias camadas de neurônios

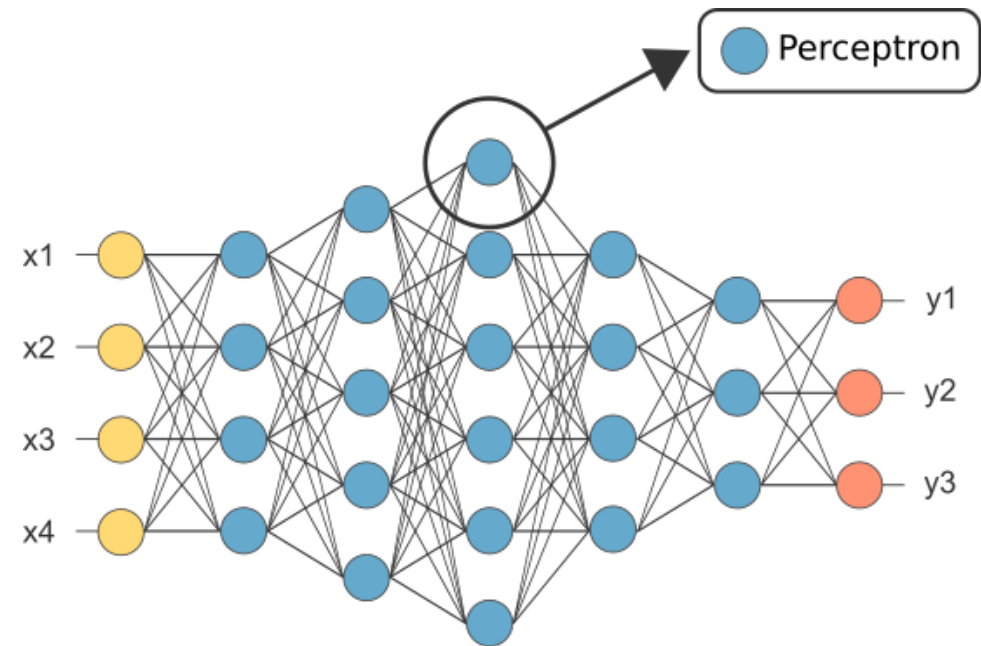
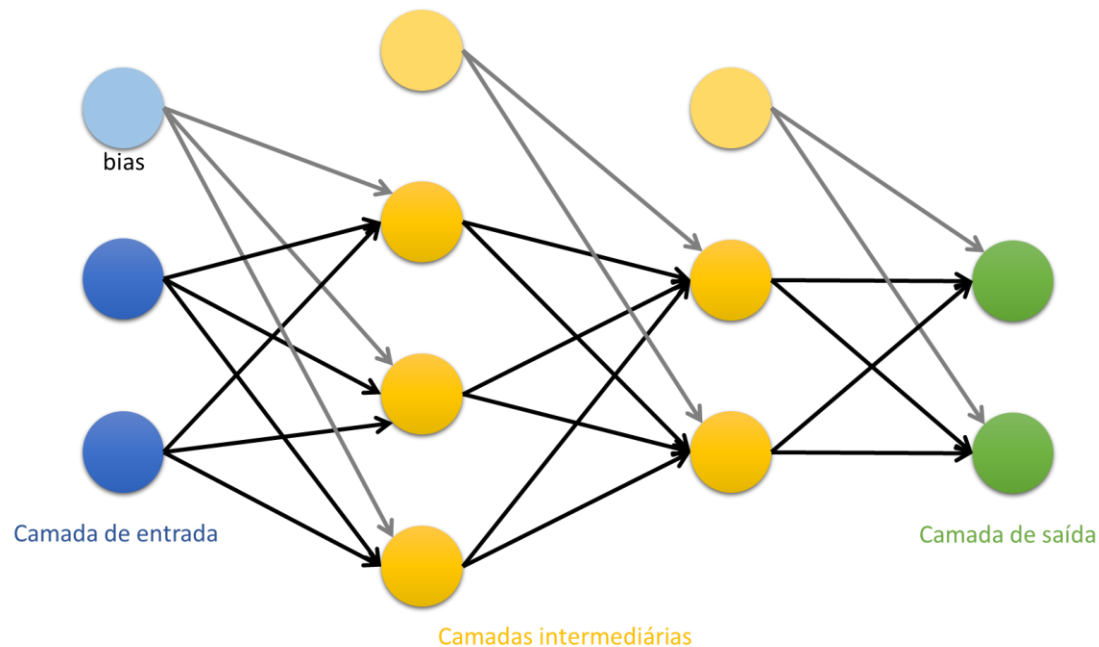


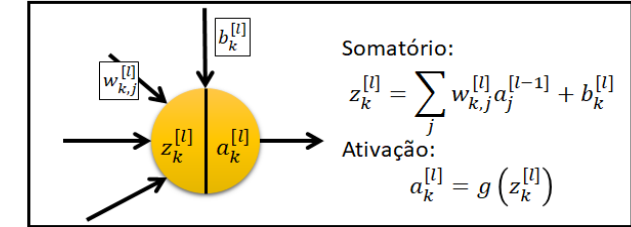
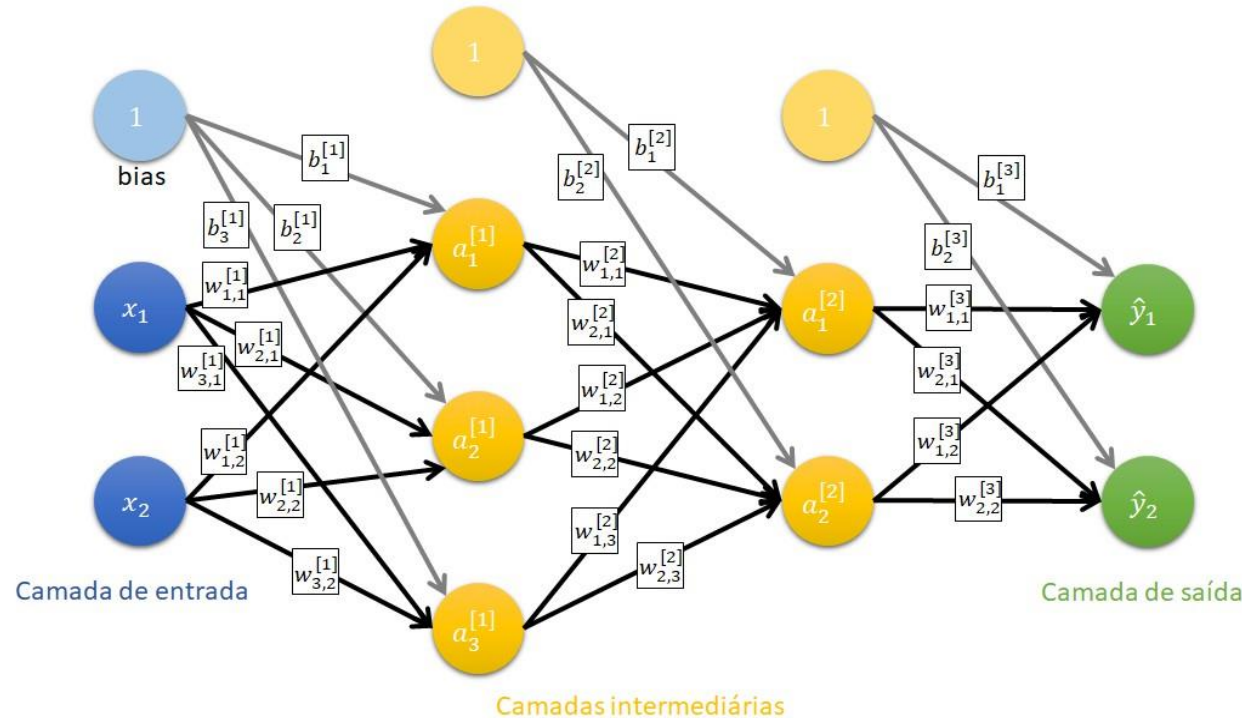
Imagem retirada de:
<https://www.oficinadanet.com.br/tecnologia/25007-o-que-sao-as-redes-neurais-artificiais>

Tipos de Camadas de uma RNA



Exemplo de RNA com $L = 3$ camadas intermediárias
(veja que a camada de saída entra na conta)

Variáveis de uma RNA



x_i : entrada

$\hat{y}_j$: saída

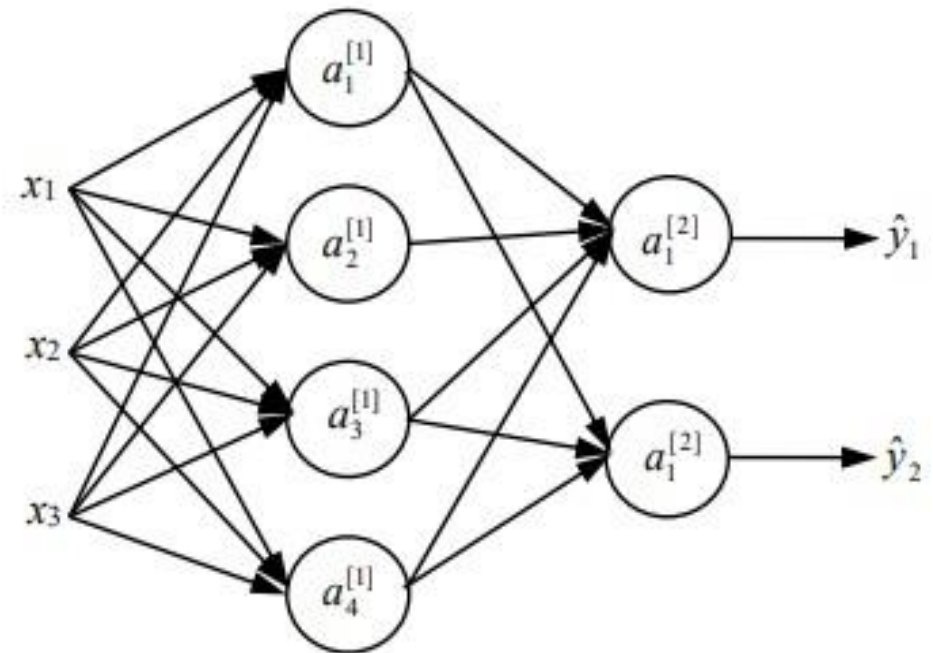
$a_i^{[j]}$: valor do nó da camada j

$w_{p,q}^{[j]}$: peso entre nós p e q da camada j

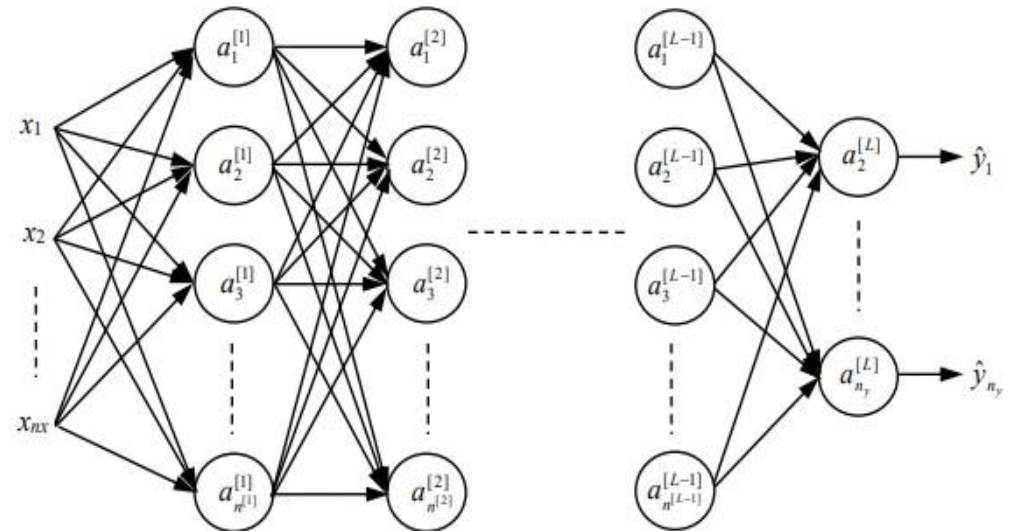
$b_i^{[j]}$: peso do bias da camada j

g : função de ativação

Avaliação de uma RNA



Avaliação de uma RNA



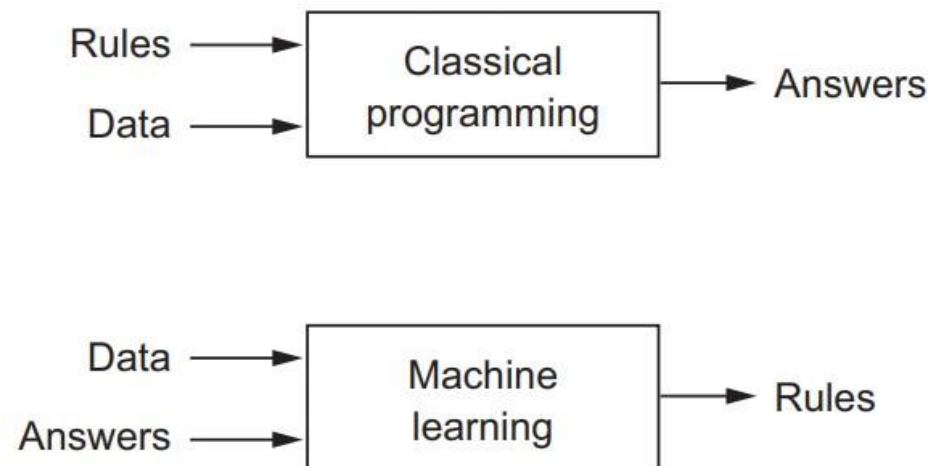
Quais são as limitações intrínsecas de uma RNA?

-
-
-

Treinamento Supervisionado e Não Supervisionado

Como treinar uma RNA?

- **Treinamento é o processo de definir os pesos da rede**
- Uma RNA deve ser treinada para que ela consiga mapear um problema
 - Seja de regressão ou de classificação
- O *aprendizado supervisionado* de uma RNA exige um conjunto de dados rotulados e classificados.
- No *aprendizado não-supervisionado*, a RNA encontra padrões em dados não rotulados



Chollet, F. Deep Learning with Python, Manning Shelter Island Editor, 2018.

Como treinar seu dragão?



Tipos de treinamento

Supervisionado

Não-
supervisionado

Auto-
supervisionado

Por reforço

Treinamento Supervisionado



Cachorro



Gato

- Dados de entrada e de saída são conhecidos
- Problemas de classificação
- Problemas de regressão

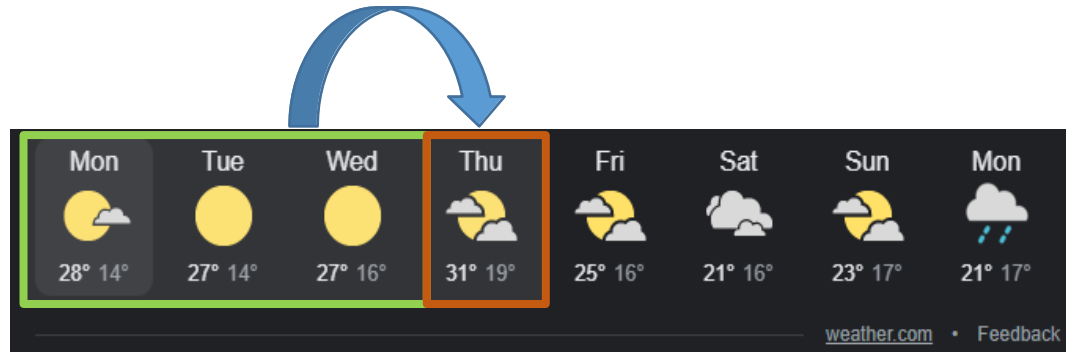
Fotos de Wikimedia commons

Treinamento Não-Supervisionado



- Somente dados de entrada são conhecidos
- Redução de dimensões
- Clusterização

Treinamento Auto-Supervisionado



- As saídas (categorias) não são definidas por humanos, mas sim, extraídas dos dados
- Predição da próxima palavra num texto
- Identidades com ocultamento
- Séries temporais

Aprendizado por Reforço



- Treinamento faz rede buscar uma recompensa
- Pontuação em um jogo
- Robótica

Qual o objetivo de um treinamento de um modelo?

O objetivo final de um processo de um processo de treinamento de aprendizado de máquina é.....

Qual o objetivo de um treinamento de um modelo?

O objetivo final de um processo de treinamento de aprendizado de máquina é.....

..... fazer o treinador feliz

Bonus: Boom or Doom

O estado da IA em 2026

IA: Boom ou Doom?

Encerramento

Aula 1 – Introdução às Redes Neurais Artificiais

Histórico
Inteligência
Artificial

Redes Neurais
Artificiais

Estrutura de
uma RNA

Treinamento
supervisionado
e não-
supervisionado

Próxima aula

Perceptrons

Classificação
binária

Gradiente
descendente

Enunciado
Trabalho 1



PECE
Engenharia de
Computação

OBRIGADO

IAD-014 Redes Neurais

aula 1

Introdução às Redes

Neurais Artificiais

Claudio Pinhanez

(a partir de materiais criados por Marlon Sproesser Mathias)